

COMP 426 - Project Report

Game of Life using OpenCL and OpenGL interoperability

Patrick Deniso (40194944)

November 2025

1. Problem Statement

The objective of this project was to implement a multi species, two dimensional Game of Life simulation using both CPU and GPU compute devices through OpenCL.

The project required the creation of a hybrid architecture in which program execution is divided between CPU and GPU, and OpenGL is only used for rendering. This differs from Assignment 3, where all computation occurred on the GPU.

So, the problem involves three design questions:

1. Where the computation occurs (CPU vs GPU)
2. How the computation is parallelized (SIMD vs MIMD)
3. How the simulation data is transferred efficiently to the renderer (OpenGL) without CPU bottlenecks

2. Project Development

2.1 Program Flow

The structure of the program followed a clear pipeline:

1. Initialization
 - We create OpenGL context through GLFW
 - Load and compile OpenGL shaders (for fullscreen quad rendering)
 - Create an OpenGL texture used to display the current simulation state
 - Initialize OpenCL: select a platform, choose devices (cpu and gpu), and create a shared OpenCL context
 - Load kernels from `kernels.cl`
 - Allocate two OpenCL buffers (`dCurr`, `dNext`) representing the current and next simulation state across all species.
 - Initialize grid data with randomized species

2. Main Loop

- Execute the CPU assigned portion of life update
- Execute the GPU assigned portion of life update
- Synchronize both devices
- Swap the two `dCurr` and `dNext` buffers
- Acquire the OpenGL texture for OpenCL
- Run the GPU blit kernel to write RGBA pixels into the texture
- Release the texture and draw it to the screen using OpenGL
- Measure frame time for performance analysis

3. Shutdown

2.2 CPU - GPU Pipeline

The program uses a CPU and GPU pipeline where each device performs a portion of the GoL computation. The host coordinates these tasks and enforces synchronization. The pipeline is:

```
[CPU life update kernel] -> [GPU life update kernel] -> [GPU blit kernel] -> [OpenGL render]
```

The CPU and GPU operate on different sets of species so that they don't write to the same region of memory. This makes the memory access race free. After both kernels complete, the host swaps buffers so that the next iteration use updated data.

The pipeline also uses OpenCL & OpenGL interoperability to avoid copying data across CPU/GPU (and is therefore significantly more efficient). The OpenCL blit kernel writes directly into a GL texture, and OpenGL immediately renders it.

3. Computation Threads Design

3.1 CPU kernel

The CPU kernel is an OpenCL kernel executed on a CPU device queue. Although CPUs are optimized for MIMD workloads and have fewer hardware threads, OpenCL still expresses this computation through an `NDRange`. The CPU then executes these work items in a chunked manner.

Key properties:

- MIMD architecture means branches are inexpensive
- Good for species that require irregular computation

- Useful for demonstrating OpenCL portability, even though performance is worse than GPU

3.2 GPU kernel

The GPU kernel performs the same GoL logic but on GPU assigned species. GPUs excel at this because each cell update is independent and uniform. The `NDRange` is `(width, height)`, producing one work item per cell (therefore SIMD friendly)

Key properties:

- Massive parallelism (hundreds of thousands of work items)
- Memory access patterns coalesced because each work item accesses neighbors in predictable patterns
- Requires careful avoidance of branching when possible

3.3 OpenCL - OpenGL blitting

At this point a dedicated GPU kernel converts each cell species value into an RGBA fragment and writes it directly into an OpenGL texture via a shared OpenCL memory object (`clCreateFromGLTexture`). This removes any copying on the CPU side

The pipeline for the blit step is as follows:

1. Acquire the GL texture (`clEnqueueAcquireGLObjects`)
2. Run the blit kernel
3. Release it back to GL (`clEnqueueReleaseGLObjects`)
4. OpenGL renders a fullscreen quad sampling the texture

4. Analysis

4.1 Measuring Performance

To measure performance, frame time and FPS were recorded using `std::chrono::steady_clock`. For each frame, we record:

- timestamp before simulation
- timestamp after rendering
- $\text{fps} = 1 / \text{frame_time}$

4.2 Interpretation

- GPU performance is substantially better due to SIMD execution and high memory bandwidth
- CPU performance is not as good because of the small number of cores and lack of hardware level parallelism

5. Portability Discussion

The project was tested on Linux (on a VM) and macOS.

Successes

- The same OpenCL kernels ran unmodified on both CPU and GPU queues.
- The same host code compiled on macOS and Linux.
- GPU kernels behaved identically across devices.

Challenges

- macOS deprecated CPU OpenCL drivers, so CPU kernels are not exposed at all and cannot run
- My Linux system required installing `pocl` for CPU execution

Conclusion

Portability does exist at the API level in theory, but varies in practice depending on platform support. GPU kernels were portable across Linux and macOS, but CPU OpenCL support was inconsistent on macOS.